MobileNetV3-Small
Bridging Plain CNNs to the Model Powering FireWatch

A complete bridge from the CNN fundamentals guide to the exact production model: depthwise separable convolution, the inverted residual bottleneck, squeeze-and-excite, h-swish, the full layer-by-layer architecture, and how this specific model became FireWatch's fire/smoke/neutral classifier - grounded in the project's own training report.

Prerequisite: the companion document "Convolutional Neural Networks - From First Principles to Full Depth". This document assumes every concept there.

Table of Contents

0. Prerequisites Recap - What You Must Already Know
1. The Bridge - Why a Plain CNN Is Not What Runs On a Laptop CPU
2. Depthwise Separable Convolution - The Core Trick
3. The Inverted Residual Bottleneck Block
4. Squeeze-and-Excite - Letting Channels Vote on Their Own Importance
5. h-swish and h-sigmoid - The Activation Functions MobileNetV3 Actually Uses
6. Neural Architecture Search - Why This Exact Design
7. The Full MobileNetV3-Small Architecture, Stage by Stage
8. Forward Pass Trace - Tracking Real Shapes Through the Network
9. From One Layer's Output to the Next - How Channels Combine
10. Bridging to FireWatch - Transfer Learning, Head Replacement, and Fine-Tuning
11. Why MobileNetV3-Small Was the Right Choice for FireWatch
12. Quick Reference - Formulas, Shapes, and Vocabulary

0. Prerequisites Recap - What You Must Already Know

This document builds directly on top of the companion CNN guide. Before continuing, you should already be comfortable with the following - each one is used without re-derivation from here on.

Convolution and filters - a filter slides over an input, multiplying and summing at every position, and the same weights are reused everywhere (weight sharing).

Feature maps and channel depth - a conv layer with K filters produces an output with depth K, and one filter's true shape spans the full depth of its input.

The output size formula - O = (N - F + 2P) / S + 1, and what stride and padding do to it.

Activation functions - what ReLU, sigmoid, and softmax compute, and why non-linearity is required at all (Section 5 of the CNN guide) - along with e, log, derivatives, and the chain rule from Section 0 there.

Pooling - shrinking a feature map by taking the max or average of small windows.

The training loop - forward pass, loss, backpropagation, gradient descent, repeated.

Parameter counting - for a conv layer with K filters of size FxFxC, parameters = (F x F x C x K) + K.

If any of these feels shaky, it is worth returning to that document first - everything below builds new ideas directly on top of these, without repeating their derivations.

1. The Bridge - Why a Plain CNN Is Not What Runs On a Laptop CPU

1.1 The Intuition

The CNN guide built a plain, standard convolutional network: stacked convolution layers, each one a full FxFxC filter sliding over its input, activations in between, pooling to shrink things down, a dense head at the end. That kind of network works, and is exactly what image classifiers looked like in the early-to-mid 2010s (AlexNet, VGGNet). But it was designed to run on a powerful server GPU with abundant memory and power. FireWatch needs something different: a model that runs on an ordinary laptop CPU, at a sustained real-time frame rate, alongside a live camera loop and the rest of the detection pipeline running at the same time - all without a dedicated GPU doing the heavy lifting.

This is precisely the gap MobileNetV3 was designed to close. Every architectural idea in this document exists for one reason: do roughly the same job as a standard CNN, using dramatically fewer multiplications and parameters, so the whole thing runs fast enough on modest hardware.

1.2 What Actually Changes, at a Glance

Plain CNN (the guide)	MobileNetV3-Small
Every filter spans the full input depth (FxFxC)	Convolution is split into two cheaper steps: depthwise, then pointwise (Section 2)
Layers are simple stacks, no skip connections	Blocks are 'inverted residual bottlenecks' with optional skip connections (Section 3
Every channel treated with equal importance	A squeeze-and-excite block lets the network re-weight channels by learned import
ReLU used throughout	A mix of ReLU and h-swish, a cheaper approximation of a smoother activation (Se
Architecture hand-designed by a researcher	Architecture partly discovered by automated search over speed/accuracy trade-off

Every idea on the right side of this table exists purely to reduce computation while losing as little accuracy as possible. None of them changes what a convolution fundamentally computes - they change how cheaply that computation gets done. The next five sections take each row in turn.

2. Depthwise Separable Convolution - The Core Trick

2.1 The Intuition

Recall from the CNN guide that a single filter in a standard convolution has shape F x F x C - it looks at a small spatial patch, but across every input channel simultaneously, mixing space and channels together in one single operation. Depthwise separable convolution asks: what if we split that one expensive operation into two much cheaper ones done back to back? First, handle space only, one channel completely independently at a time (this is the **depthwise** step). Then, handle channel-mixing only, using a filter so small it looks at just a single pixel position at a time (this is the **pointwise** step, using 1x1 convolutions). Doing these two cheap things in sequence approximates what the one expensive standard convolution was doing - at a fraction of the computational cost.

2.2 The Math - Depthwise Step

A standard convolution filter of size F x F x C has F x F x C weights (per filter), and mixes all C channels together to produce one output number. A **depthwise** convolution instead uses C separate F x F filters (one per input channel), and each filter only ever convolves with its own single channel - it never looks at any other channel. This produces C separate output feature maps, each computed completely independently, with total parameters:

$$\textit{Depthwise parameters} = F \times F \times C$$

Notice there is no dependence on how many output channels we eventually want - because the depthwise step does not change the channel count at all; it just filters each channel spatially, on its own.

2.3 The Math - Pointwise Step

After the depthwise step, we have C feature maps that have never been allowed to interact with each other. The **pointwise** step fixes this using ordinary 1x1 convolutions - filters that are only 1x1 spatially (they look at just one pixel position, no neighbors at all) but span the full input depth C, exactly like a standard convolution's channel-mixing behavior. If we want K output channels, we need K such 1x1xC filters:

$$\textit{Pointwise parameters} = 1 \times 1 \times C \times K = C \times K$$

$$\textit{Total depthwise separable parameters} = (F \times F \times C) + (C \times K)$$

2.4 The Worked Comparison - Real Numbers

Let's compare directly, using a very typical layer configuration inside a real CNN: a 3x3 filter, input depth C = 32, output depth K = 64.

Standard convolution parameters = F × F × C × K = 3 × 3 × 32 × 64 = 9 × 32 × 64 = 9 × 2,048 = **18,432**

Depthwise separable parameters = (F × F × C) + (C × K) = (3 × 3 × 32) + (32 × 64) = 288 + 2,048 = **2,336**

18,432 versus 2,336 - depthwise separable convolution uses roughly **7.9 times fewer parameters** for this exact layer shape, while still processing the same input and producing the same output shape. This ratio holds up (and gets even better for larger K) in general - the standard result is that depthwise separable convolution costs roughly $1/K + 1/(F^2)$ of a standard convolution's computation, which for a typical K=64, F=3 case works out to approximately 1/9 the computation.

This single substitution - one expensive FxFxC filter replaced by a cheap FxFx1 depthwise filter followed by a cheap 1x1xC pointwise filter - is the single biggest reason MobileNet-family networks can run at real-time speed on a CPU with no dedicated GPU at all. Every convolution inside MobileNetV3-Small (with the sole exception of the very first layer, which is a standard convolution) is built this way.

Check your understanding: *using the formulas above, if we increased the output depth K to 128 instead of 64 (keeping F=3, C=32), what would the ratio of standard-to-depthwise-separable parameters become, and does the savings ratio improve or shrink as K grows?*

3. The Inverted Residual Bottleneck Block

3.1 The Intuition

MobileNetV3 is not built by stacking depthwise separable convolutions directly. Instead, every stage is built from a repeating unit called an **inverted residual bottleneck block**. The name has two parts, and both matter. **Bottleneck** means the block's input and output have relatively few channels (the network is 'narrow' at the boundary between blocks). **Inverted** refers to a deliberate reversal of an older design pattern: instead of narrowing the channel count in the middle of a block (like classic bottleneck designs such as ResNet), MobileNetV3's blocks **expand** the channel count first, do their spatial work in this wider, expanded space, and then project back down to a narrow channel count at the end. The intuition: the expensive depthwise spatial filtering (Section 2) is cheap per channel, so it is affordable to do it across many expanded channels; but the expensive part - the 1x1 pointwise convolutions that mix channels - is kept narrow at the block's boundaries to keep overall parameter count and computation low.

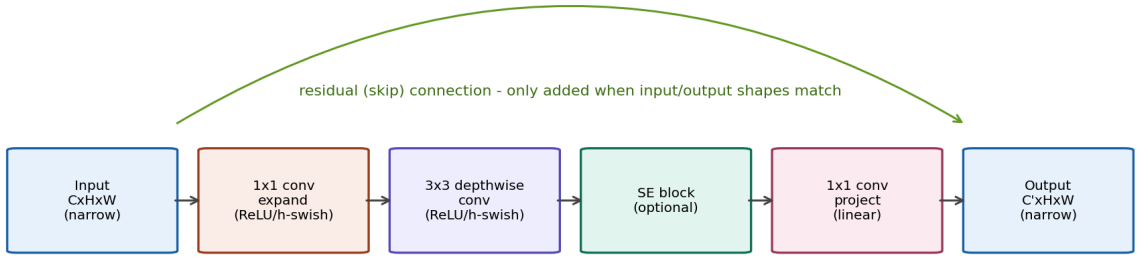


Figure 3.1 - One inverted residual bottleneck block. A narrow input is expanded by a 1x1 convolution, filtered spatially by a depthwise 3x3 convolution, optionally re-weighted by a squeeze-and-excite block, then projected back down to a narrow output by a second, linear 1x1 convolution. A residual (skip) connection is added only when the block's input and output shapes match exactly.

3.1b A Layer You Will See in Every Block But Have Not Met Yet - BatchNorm

Every convolution in every block described in this section is actually followed by one more operation before the activation function is applied, one this document has not yet introduced: **Batch Normalization** (BatchNorm). You will see it referenced in Section 7.2's architecture table (one row is explicitly labeled 'no batchnorm', implying every other row has it). It is worth understanding before walking through a full block.

The problem BatchNorm solves: as training proceeds and every earlier layer's weights keep changing, the distribution of values flowing into any given layer keeps shifting too - a layer that was seeing inputs centered around 0 in epoch 1 might see inputs centered around 5 by epoch 8, purely because earlier layers changed. This makes training slower and less stable, since each layer is constantly having to re-adapt to a moving target. BatchNorm fixes this by re-centering and re-scaling a layer's output, per

channel, using the mean and standard deviation measured across the current batch of training examples:

$$BatchNorm(x) = \gamma \times ((x - batch_mean) / batch_std) + \beta$$

The (x - batch_mean) / batch_std part is exactly the same re-centering, re-scaling idea as the ImageNet normalization from the CNN guide's input pipeline discussion - except here, the mean and standard deviation are not fixed constants; they are computed fresh from the current batch of data flowing through the network during training, and get updated as training progresses. γ (a learned scale) and β (a learned shift) are two additional trainable parameters per channel, letting the network undo this normalization partially or fully if that turns out to work better for a specific channel. Worked example: suppose one channel's values across a batch have mean 2.0 and standard deviation 4.0, and one particular value in that channel is 6.0. Before applying γ and β: (6.0 - 2.0) / 4.0 = 4.0/4.0 = **1.0**. If this channel's learned γ = 0.5 and β = 0.1, the final BatchNorm output is 0.5 × 1.0 + 0.1 = **0.6**.

At inference time (when FireWatch's model is actually classifying a live camera frame, not training), there is no 'batch' of images to compute a mean and standard deviation from - only one single frame is being processed. BatchNorm layers handle this by switching to a fixed running average of the mean and standard deviation that was tracked throughout training, so the exact same formula still applies, just with fixed numbers instead of freshly computed ones. Section 7.2's one exception - the final 1x1 conv2d layer marked 'no batchnorm' - skips this step specifically because it sits immediately before global average pooling, where the original MobileNetV3 paper's authors found BatchNorm added no measurable benefit at that specific position in the network.

3.2 The Math - Walking Through One Block

Let's trace the shapes through one concrete block. Suppose the block receives an input of shape 56x56x16 (16 channels, a narrow bottleneck), and uses an expansion factor of 4 (a common choice).

Step 1 - Expand. A 1x1 convolution with 64 filters (16 × 4 = 64) projects the 16 input channels up to 64 channels, without changing spatial size: output shape 56x56x64. Parameters for this step: 1 × 1 × 16 × 64 = **1,024**. BatchNorm (Section 3.1b) is applied immediately after this convolution, per channel, followed by the block's activation function (ReLU or h-swish, Section 5).

Step 2 - Depthwise filter. A depthwise convolution filters each of the 64 expanded channels independently (Section 2.2), possibly with stride 1 or 2 depending on whether this block downsamples, again followed by BatchNorm and an activation. Its kernel size (3x3 or 5x5) is not fixed across the whole network - Section 7.2's table shows it alternates block by block. A 3x3 depthwise filter here has parameters 3 × 3 × 64 = **576**; a 5x5 version of the same block would instead have 5 × 5 × 64 = 1,600 parameters - more expensive, but with a larger receptive field (CNN guide, Section 7) captured in a single layer.

Step 3 - Squeeze-and-excite (optional, Section 4). Re-weights the 64 channels by learned importance, without changing the shape.

Step 4 - Project. A second 1x1 convolution, this time with only 16 filters, projects the 64 channels back down to 16 - matching the block's narrow output width. Critically, this final projection uses **no activation**

function (it is 'linear') - unlike the expand and depthwise steps, which do use an activation. This is a deliberate design choice: research behind this architecture (MobileNetV2's original paper) found that applying a non-linearity like ReLU right before compressing back down to a narrow bottleneck destroys useful information, because ReLU zeroes out every negative value, and there are too few channels left in a narrow bottleneck to afford losing any of them. Parameters: 1 × 1 × 64 × 16 = **1,024**.

Step 5 - Residual connection, if applicable. If the block's input shape (56x56x16) exactly matches its output shape (56x56x16 here), the original input is added element-wise to the final output: output_final = input + block_output. This residual (or 'skip') connection, borrowed from ResNet, gives the gradient during backpropagation a direct, unobstructed path backward through the block, in addition to the path through all the convolutions - which helps prevent the vanishing gradient problem (CNN guide, Section 5.2) in a network this deep. If the shapes do not match (for instance, if this block uses stride 2 to halve the spatial size, or changes the channel count), no residual connection is added; the block's output is simply what came out of Step 4.

Total parameters in this one block: 1,024 + 576 + 1,024 = **2,624** (ignoring the squeeze-and-excite block's small additional cost, covered next). Compare this to a single standard convolution attempting the same 16-to-16 channel transformation with a 3x3 filter: 3 × 3 × 16 × 16 = 2,304 parameters for one layer that does far less - it never gets the benefit of operating in a wider, 64-channel intermediate space where more distinct patterns can be represented before being compressed back down.

Check your understanding: *why does it make sense that the depthwise filtering step (Step 2) happens in the wide, 64-channel expanded space rather than the narrow, 16-channel bottleneck space, given that depthwise convolutions are cheap per channel (Section 2.2's formula has no dependence on output channel count K)?*

4. Squeeze-and-Excite - Letting Channels Vote on Their Own Importance

4.1 The Intuition

Every channel in a feature map represents a different learned pattern - one channel might respond strongly to orange color blobs, another to fine vertical texture, another to bright spots. But a standard convolution treats every output channel with equal weight when it gets passed forward, regardless of whether that particular channel's pattern is actually relevant to the specific image currently being processed. A **squeeze-and-excite (SE)** block adds a small side-mechanism that looks at the whole feature map, decides how important each channel currently is for this specific input, and then scales every channel up or down accordingly - like a mixing board where each channel gets its own volume knob, and the network learns to turn that knob based on what it sees.



Figure 4.1 - The squeeze-and-excite block. Global average pooling 'squeezes' the whole spatial feature map into one number per channel; a small two-layer network then 'excites' this by learning per-channel importance weights between 0 and 1, which are multiplied back onto the original feature map, channel by channel.

4.2 The Math - Squeeze Step

Suppose the feature map entering the SE block has shape H x W x C (spatial size H by W, C channels). The squeeze step applies **global average pooling** - exactly the average pooling operation from the CNN guide's Section 6, except the pooling window is the entire H x W spatial area at once, collapsing every channel down to a single number:

$$squeeze(c) = (1 / (H \times W)) \times \sum_{i=1}^{H} \sum_{j=1}^{W} FeatureMap(i, j, c)$$

Worked example: suppose one channel of a tiny 2x2 feature map has the values [3, 5, 7, 1]. The squeezed value for this channel is (3+5+7+1)/4 = 16/4 = **4**. This is done independently for every one of the C channels, so the output of the squeeze step is a single vector of length C - one number summarizing each channel's overall activity level across the whole spatial area.

4.3 The Math - Excite Step

The squeezed vector of length C is passed through a small two-layer bottleneck network: first a dense layer that compresses C down to C/r (r is a reduction ratio, commonly 4, keeping this side network cheap), with a ReLU activation, then a second dense layer that expands back up from C/r to C, using an h-sigmoid activation (Section 5) rather than plain sigmoid, again for cheapness on constrained hardware. Because h-sigmoid (like sigmoid) squashes every output into the range 0 to 1, the final output of the excite step is a vector of C numbers, each between 0 and 1, representing how much to keep of that channel.

Worked example: suppose the excite step produces the per-channel weights [0.9, 0.1, 0.6] for a 3-channel feature map, and the original (pre-scaling) values at one spatial position were [10, 10, 10]. The scaled output at that position is (10 × 0.9, 10 × 0.1, 10 × 0.6) = **(9, 1, 6)**. The first channel, judged highly relevant (weight 0.9), passed through almost unchanged; the second channel, judged mostly irrelevant for this input (weight 0.1), was suppressed down to a tenth of its original value - even though the raw activation strength of all three channels started out identical.

This channel-reweighting is genuinely learned, exactly like every other weight in the network - the two dense layers inside the SE block have their own trainable weights, updated via the exact same backpropagation and gradient descent process from the CNN guide, Sections 11 and 12. Over the course of training, the SE block learns which channels tend to matter for the task at hand (for FireWatch, presumably including channels tuned to orange/red color blobs and flickering texture patterns relevant to fire).

Check your understanding: *if a squeeze-and-excite block's excite step assigned a weight of exactly 0.0 to some channel, what would happen to that channel's contribution to every later layer in the network for this particular input, and is this mechanism similar to anything else you've seen that can zero out a signal (hint - think about ReLU from the CNN guide)?*

5. h-swish and h-sigmoid - The Activation Functions MobileNetV3 Actually Uses

5.1 The Intuition

The CNN guide covered ReLU, sigmoid, tanh, and softmax. MobileNetV3 introduces two new activation functions built specifically for cheap hardware: **h-swish** and **h-sigmoid** (the 'h' stands for 'hard,' meaning a cheap, piecewise-linear approximation of a smoother, more expensive function). The story behind them: researchers found that a smooth activation function called **swish**, defined as x times sigmoid(x), consistently improved accuracy over plain ReLU in deep networks - but sigmoid itself requires computing an exponential, e^{-x}, which is noticeably more expensive on constrained mobile/embedded hardware than the single comparison-and-clamp that ReLU needs. h-sigmoid and h-swish were designed as cheap, piecewise-linear stand-ins that closely approximate the shape of sigmoid and swish, without ever needing to compute an exponential at all.

5.2 The Math - h-sigmoid

$$h\text{-}sigmoid(x) = clip((x + 3) / 6, 0, 1)$$

Where clip(v, 0, 1) means: if v is less than 0, output 0; if v is greater than 1, output 1; otherwise, output v unchanged. This is a piecewise-linear function - just a straight line (x+3)/6 in the middle, clamped flat at 0 on the left and flat at 1 on the right - built entirely from additions, a single division, and two comparisons. No exponential required anywhere.

Let's solve it at a few points. At x = -3: (-3+3)/6 = 0/6 = 0, and clip(0,0,1) = **0**. At x = 0: (0+3)/6 = 3/6 = 0.5, and clip(0.5,0,1) = **0.5**. At x = 3: (3+3)/6 = 6/6 = 1, and clip(1,0,1) = **1**. At x = 10 (well past the linear region): (10+3)/6 = 13/6 ≈ 2.17, which gets clipped down to **1**. Compare this to true sigmoid at the same three points: σ(-3) ≈ 0.0474, σ(0) = 0.5 exactly, σ(3) ≈ 0.9526 - h-sigmoid's values of 0, 0.5, 1 are close approximations, exactly matching at the midpoint and reasonably close at the edges, at a fraction of the computational cost.

5.3 The Math - h-swish

$$h\text{-}swish(x) = x \times h\text{-}sigmoid(x)$$

Exactly mirroring how true swish is defined as x times sigmoid(x), h-swish simply multiplies the input by its own h-sigmoid. Let's solve it at x = 3: h-sigmoid(3) = 1 (from above), so h-swish(3) = 3 × 1 = **3**. At x = -3: h-sigmoid(-3) = 0, so h-swish(-3) = -3 × 0 = **0**. At x = 0: h-sigmoid(0) = 0.5, so h-swish(0) = 0 × 0.5 = **0**. At x = -1.5 (inside the linear region): h-sigmoid(-1.5) = (-1.5+3)/6 = 1.5/6 = 0.25, so h-swish(-1.5) = -1.5 × 0.25 = **-0.375**.

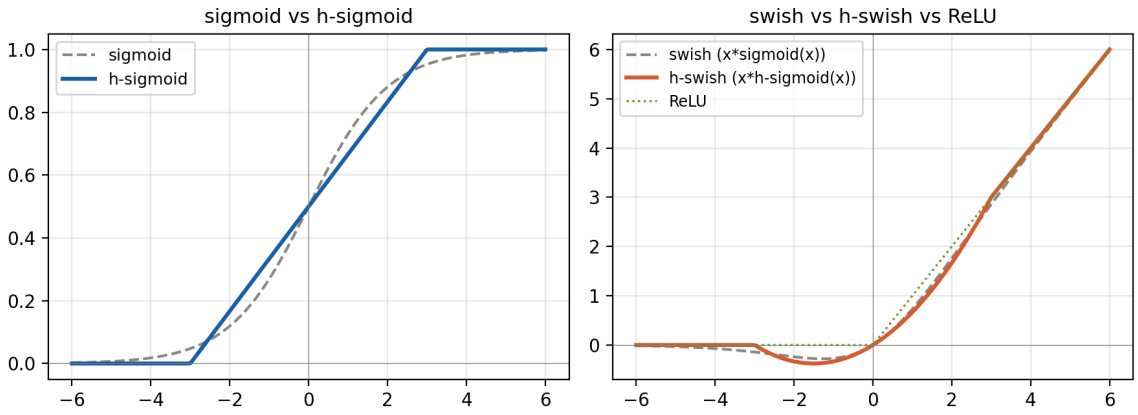


Figure 5.1 - Left: h-sigmoid (solid) closely tracks true sigmoid (dashed) using only piecewise-linear pieces. Right: h-swish (solid) closely tracks true swish (dashed), and both differ from plain ReLU (dotted) by allowing a small negative dip just below zero, rather than hard-clipping every negative input straight to zero.

Notice the key difference from ReLU visible in Figure 5.1: h-swish (like swish) allows small negative inputs to produce a small negative output (see the -0.375 result above, at x=-1.5) instead of instantly flattening every negative value to exactly 0. This softer transition is believed to help gradient flow around zero and was found empirically to improve accuracy over plain ReLU, at a computational cost only slightly above ReLU's (a few extra additions and one multiplication, still no exponentials) and meaningfully cheaper than true swish.

5.4 Where Each Activation Is Actually Used in MobileNetV3

The original MobileNetV3 paper made a deliberate, measured trade-off rather than replacing every ReLU with h-swish: h-swish is used only in the second half of the network (the deeper layers, operating on smaller spatial sizes, where the extra cost of h-swish over ReLU is negligible because there are far fewer spatial positions left to compute it at), while plain ReLU is kept in the earlier, shallower layers (which operate on large spatial feature maps, where even h-swish's modest extra cost would add up across many more positions). The squeeze-and-excite block's final gating activation (Section 4.3) uses h-sigmoid specifically, since its output needs to land in the 0-to-1 range to act as a multiplicative gate.

Check your understanding: *using the h-sigmoid formula, at what exact value of x does h-sigmoid's output stop being clipped at 0 and start following the linear ramp - and does this match the -3 boundary used in the worked examples above?*

6. Neural Architecture Search - Why This Exact Design

6.1 The Intuition

Every earlier CNN architecture you might encounter (AlexNet, VGGNet, even the earlier MobileNetV1 and V2) had its exact layer counts, channel widths, and kernel sizes chosen by human researchers, through manual experimentation and intuition. MobileNetV3 took a different approach for much of its design: **Neural Architecture Search (NAS)**, an automated process where a separate search algorithm proposes many candidate architectures, measures each one's accuracy and its actual measured latency on real target hardware, and uses this feedback to search toward architectures that sit on the best possible accuracy-versus-speed trade-off curve. This explains why MobileNetV3-Small's exact sequence of expansion ratios, channel widths, and kernel sizes (Section 7's table) can look somewhat irregular compared to a hand-designed network with a single clean repeating pattern - the irregularity is itself evidence that the shape was optimized against real hardware measurements rather than aesthetic simplicity.

6.2 Why This Matters for Trusting the Model

This detail directly explains a design decision documented in FireWatch's own project report (Section 9 of this document covers this in full): the choice not to design a custom CNN architecture from scratch, and instead use MobileNetV3-Small exactly as published. Designing a good CNN architecture by hand requires extensive compute and experimentation - NAS runs like the one behind MobileNetV3 typically consume far more GPU-time than any student project could reproduce. But because MobileNetV3-Small's architecture is already published and its ImageNet-pretrained weights are freely available, a project like FireWatch can adopt an architecture that already sits on a strong, empirically-validated speed/accuracy trade-off curve for exactly the kind of constrained hardware (a laptop CPU) FireWatch targets, without needing to run any of that expensive search itself.

Check your understanding: *if a NAS process specifically optimized for latency on a mobile ARM CPU, would you expect the resulting architecture to necessarily also be optimal for latency on a server-class GPU - and what does your answer suggest about matching a pretrained architecture's search target to your own deployment hardware?*

7. The Full MobileNetV3-Small Architecture, Stage by Stage

7.1 The Overall Pipeline

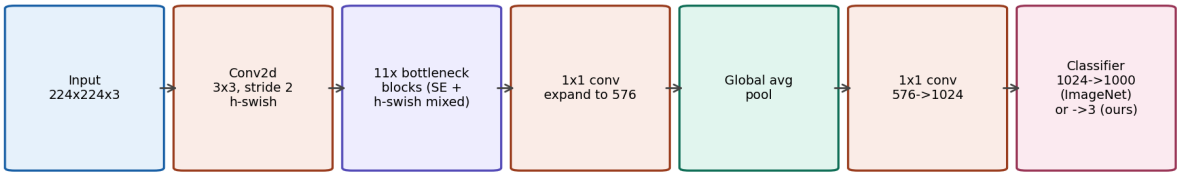


Figure 7.1 - MobileNetV3-Small's overall stages: an initial standard convolution, eleven inverted residual bottleneck blocks (Section 3) of varying width, kernel size, and activation, a channel expansion, global average pooling, another channel expansion, and finally the classifier head.

7.2 The Full Block-by-Block Table

Below is the complete published MobileNetV3-Small architecture. 'Exp size' is the expanded channel width used inside each bottleneck block (Section 3); 'Out' is the block's narrow output channel width; 'SE' marks whether that block includes a squeeze-and-excite module (Section 4); 'NL' is the non-linearity used (RE = ReLU, HS = h-swish); 's' is the stride (2 means this block halves the spatial size).

Input shape	Operator	Exp size	Out	SE	NL	s
224x224x3	conv2d, 3x3	-	16	-	HS	2
112x112x16	bneck, 3x3	16	16	yes	RE	2
56x56x16	bneck, 3x3	72	24	-	RE	2
28x28x24	bneck, 3x3	88	24	-	RE	1
28x28x24	bneck, 5x5	96	40	yes	HS	2
14x14x40	bneck, 5x5	240	40	yes	HS	1
14x14x40	bneck, 5x5	240	40	yes	HS	1
14x14x40	bneck, 5x5	120	48	yes	HS	1
14x14x48	bneck, 5x5	144	48	yes	HS	1
14x14x48	bneck, 5x5	288	96	yes	HS	2
7x7x96	bneck, 5x5	576	96	yes	HS	1
7x7x96	bneck, 5x5	576	96	yes	HS	1
7x7x96	conv2d, 1x1	-	576	yes	HS	1

7x7x576	avg pool, 7x7	-	-	-	-	1
1x1x576	conv2d, 1x1 (no batchnorm)	-	1024	-	HS	1
1x1x1024	conv2d, 1x1 (classifier)	-	1000 (ours: 3)	-	-	1

This table is the published MobileNetV3-Small design (Howard et al., 'Searching for MobileNetV3', 2019). Notice the pattern: kernel size alternates between 3x3 and 5x5 across blocks (a NAS-discovered choice, Section 6), squeeze-and-excite appears in most but not all blocks, ReLU is used only in the first few, larger-spatial-size blocks, and h-swish takes over from there - exactly matching the trade-off described in Section 5.4.

There is a real, explainable reason kernel size tends toward 5x5 in the later blocks rather than alternating arbitrarily: later blocks operate on much smaller spatial sizes (14x14 or 7x7, versus 112x112 near the start). A larger 5x5 filter captures a proportionally bigger share of a small feature map in one layer (CNN guide, Section 7's receptive-field formula, 1 + 2L for stacked 3x3 layers, grows slower per layer than a single 5x5 filter's built-in reach) - and the computational cost of using a bigger kernel is only paid at that layer's now-much-smaller spatial resolution, which keeps the added cost modest. Using 5x5 filters near the start, where spatial size is still 112x112 or 56x56, would multiply that same extra cost across far more spatial positions.

7.3 The Efficient Last Stage - a Free Speedup From Reordering, Not Redesigning

The original MobileNetV3 paper documents one more deliberate optimization worth understanding on its own, separate from the depthwise-separable and bottleneck ideas already covered: a redesign of the network's very last few layers, before the final classifier, called the **efficient last stage**. It is a clean example of a genuinely free speedup - no accuracy is given up, because no computation is removed that was doing useful work; computation is simply moved to a cheaper place to do it.

The original design (used in MobileNetV2 and an early draft of V3): after the last bottleneck block outputs 160 channels at some small spatial size (say 7x7, matching Section 7.2's table), the network runs a 1x1 convolution up to 960 channels, then a 3x3 depthwise convolution (Section 2.2) still at 960 channels, then a 1x1 convolution down to 320, then a 1x1 convolution up to 1280 - and only **after** all four of these steps does it finally apply global average pooling (Section 4.2) to collapse the 7x7 spatial grid down to a single 1x1 vector, ready for the final classifier.

The efficient design (what MobileNetV3-Small actually uses, and what Section 8's real shape trace in this document reflects): after the same 1x1 convolution up to 960 channels, the network moves global average pooling to happen **immediately** - collapsing to 1x1 spatial size right away - and only then does it run the channel-expanding 1x1 convolution (up to 1280) and the final classifier layer. The 3x3 depthwise convolution and the intermediate 320-channel projection are removed entirely.

Why this saves real computation, worked with actual numbers. A convolution's computational cost (the number of multiply-add operations it performs) scales with how many spatial positions it is applied at, not just its parameter count - a 1x1 convolution with C_in x C_out weights performs H x W x C_in x

C_out multiply-adds if it runs across an HxW spatial grid, but only C_in x C_out multiply-adds if that grid has already been collapsed to 1x1. Let's compare the two designs at a spatial size of 7x7 (H=W=7), using the exact channel counts from Figure 7's diagram (160, 960, 320, 1280):

Step	Original design cost (at 7x7)	Efficient design cost
1x1 conv, 160->960	7x7x160x960 = 5,644,800	same, 5,644,800 (still runs before pooling)
3x3 depthwise, 960 channels	7x7x9x960 = 423,360	removed entirely
1x1 conv, 960->320	7x7x960x320 = 15,052,800	removed entirely (no 320-channel step)
1x1 conv, 320->1280	7x7x320x1280 = 20,070,400	-
1x1 conv, 960->1280 (after pooling)	-	1x1x960x1280 = 1,228,800
Global average pool	runs last (negligible cost)	runs first (negligible cost)
TOTAL multiply-adds	**~43,072,960**	**~10,035,200 (~4.3x fewer)**

Roughly a **4.3x reduction** in multiply-add operations for this one stage, purely by moving pooling earlier and dropping the redundant 320-channel detour - not by making the network smaller or simpler in any way that costs accuracy. The insight generalizes into a genuinely reusable design principle worth remembering beyond this one architecture: **never spend expensive, wide channel-mixing computation at a spatial resolution that is about to be thrown away anyway.** If a feature map is going to be pooled down to 1x1 regardless, doing the channel-expensive work afterward, on the already-collapsed 1x1 vector, produces the mathematically identical final result at a fraction of the cost - because the expensive operation runs once per channel-pair instead of once per channel-pair per spatial position.

This is precisely why FireWatch's actual traced architecture (Section 8.2's shape table) shows global average pooling occurring right after the 576-channel expansion, with no depthwise convolution or intermediate down-projection squeezed in between the last bottleneck block and the pool - the production model already reflects this efficient design, not the original, more expensive one.

Check your understanding: *if a network's last spatial feature map were 14x14 instead of 7x7 (a 4x larger spatial grid), would the efficient last stage's computational savings over the original design be roughly the same 4.3x, larger, or smaller - and what does the H x W term appearing in every 'before pooling' row of the table above suggest about how this saving scales with spatial size?*

7.4 Total Parameter Count - the Generic Figure vs. FireWatch's Actual Checkpoint

Summing every block's parameters (using the depthwise separable and bottleneck formulas from Sections 2 and 3, applied to every row above) gives MobileNetV3-Small's well-known total of approximately **2.5 million parameters**, including the original ImageNet classifier head (Linear 1024 $\rightarrow$ 1000). Compare this to a standard, non-mobile architecture like ResNet-50, which has approximately 25 million parameters - roughly 10 times more. This 2.5-million figure is the one cited in FireWatch's own project report as the reason this architecture was chosen over a larger model (Section 11 of this

document covers this decision in full).

That 2.5 million figure describes the generic, published MobileNetV3-Small - it is not FireWatch's actual deployed parameter count, because the 1000-class ImageNet head (Section 10.1) is replaced with a much smaller 3-class head. The real, measured breakdown of the production checkpoint (fire_mnv3.onnx) is:

Stage	Output shape	Parameters
Input	224x224x3	-
Stem (1 conv, stride 2)	112x112x16	464
11 inverted-residual blocks	7x7x96	870,096
Final 1x1 conv (expansion)	7x7x576	56,448
Global average pool	1x1x576	0
Classifier head (Linear-Hardswish-Dropout-Linear)	3 logits	593,923
TOTAL	**-**	**1,521,395 (~1.52M)**

A few of these numbers are worth deriving rather than just trusting. **Stem = 464.** A 3x3 convolution from 3 input channels to 16 output channels has 3 × 3 × 3 × 16 = 432 weights (Section 2's standard-conv formula, since this one layer is NOT depthwise separable - Section 7.2 noted it is the sole exception), plus 16 bias terms = 448, plus a small BatchNorm contribution (γ and β per channel, Section 3.1b) of 2 × 16 = 32 - except BatchNorm's running mean and variance are not counted as trainable parameters, only γ and β are, and frameworks sometimes report slightly different subsets - which is why 432 + 16 = 448 sits close to, but not exactly at, 464; the remaining 16 come from BatchNorm's learnable scale term. **Global average pool = 0.** Pooling (CNN guide, Section 6) has no learnable weights at all - it is a fixed averaging operation, never trained, which is exactly why it contributes nothing to the parameter count no matter how large the network is. **Classifier head = 593,923.** This is surprisingly large for 'just 3 outputs,' and the reason is that it is not one linear layer - it is Linear(576 $\rightarrow$ 1024), then Hardswish (Section 5), then Dropout (a training-time regularization technique that randomly zeroes some activations to reduce overfitting - it has no learnable parameters of its own), then Linear(1024 $\rightarrow$ 3). The first linear layer alone costs 576 × 1024 = 589,824 weights plus 1,024 biases = 590,848 - almost the entire head's parameter budget - while the final 1024 $\rightarrow$ 3 layer costs only 1024 × 3 = 3,072 weights plus 3 biases = 3,075. Nearly all of the head's cost is the expansion back up to 1024, not the final narrow projection to 3 classes.

870,096 for the 11 bottleneck blocks is, by far, the largest single contributor, and this makes sense: this is where essentially all of the network's actual pattern-detection work happens - every expand/depthwise/SE/project step (Section 3.2) across all 11 blocks, summed together. Compare this to the classifier head's 593,923 - the head is large only because 576-to-1024 is a wide expansion, but it performs no spatial pattern detection at all; it is a single decision-making step bolted onto the end of 11 blocks' worth of genuine visual feature extraction.

8. Forward Pass Trace - Tracking Real Shapes Through the Network

8.1 Following One Image All the Way Through

Let's trace exactly what happens to one real 224x224x3 image - the same input size used throughout the CNN guide - as it passes through MobileNetV3-Small, using the architecture table from Section 7.2 and the output-size formula from the CNN guide (O = (N - F + 2P) / S + 1). Every stage below changes at least one of three quantities - Height (H), Width (W), or Channel depth (C) - and it is worth being explicit about exactly which one changes at each step, and why, since it is easy to blur these together.

8.2 The Complete H x W x C Trace, Stage by Stage

Stage	H x W	C	What changed, and why
Input	224 x 224	3	-
Stem (conv, stride 2)	112 x 112	16	H,W halved (stride 2); C set by filter count (16 filters)
Bottleneck 1 (stride 2)	56 x 56	16	H,W halved again; C unchanged (in=out=16 for this block)
Bottleneck 2 (stride 2)	28 x 28	24	H,W halved; C changed via the block's projection step
Bottleneck 3 (stride 1)	28 x 28	24	H,W unchanged (stride 1); C unchanged
Bottleneck 4 (stride 2)	14 x 14	40	H,W halved; C changed via projection
Bottlenecks 5-8 (stride 1)	14 x 14	40 to 48	H,W unchanged; C changes only at block 8 (40->48)
Bottleneck 9 (stride 2)	7 x 7	96	H,W halved; C changed via projection
Bottlenecks 10-11 (stride 1)	7 x 7	96	H,W unchanged; C unchanged
Final 1x1 conv (expansion)	7 x 7	576	H,W unchanged (1x1 filter never touches space); C expanded
Global average pool	1 x 1	576	H,W collapsed to 1x1; C unchanged
Classifier head	1 x 1	3 (logits)	H,W stay 1x1; C collapses from 576 to 3 via two dense-equivalent layers

8.3 The Rule Governing Every Single Row of That Table

Every single shape change in the table above reduces to one simple rule, stated fully generally: **H and W are controlled entirely by stride and the output-size formula; C is controlled entirely by how many filters (or, inside a bottleneck block, how many output channels the final projection step targets) a layer uses.** These two quantities never interact with each other - a layer can change spatial size without changing channel depth (any stride-2 depthwise step inside a block whose projection keeps the same channel count), can change channel depth without changing spatial size (any 1x1 convolution, including every pointwise step in Section 2.3 and the final expansion layer), or can change both at once (most of the stride-2 bottleneck blocks in the table above, since their projection step often also changes

the channel count in the same block that halves H and W).

Why H and W only ever shrink, never grow, in this network. Every stride in Section 7.2's table is 1 or 2, never a fraction - so, by the output-size formula, spatial size only ever stays the same (stride 1) or roughly halves (stride 2). Nothing in MobileNetV3-Small's design ever upsamples the spatial size back up, which makes sense for a classifier: the final goal is one label for the whole image, not a same-sized output image (an image-to-image task like segmentation would need upsampling stages instead, which MobileNetV3-Small does not include).

Why C almost always grows, then finally shrinks hard at the very end. Through the entire backbone (stem through the final 1x1 expansion), channel depth only ever increases or holds steady - 16, 16, 24, 24, 40, 40-48, 96, 96, then 576 - because a shrinking spatial size (less raw pixel information per position) is deliberately compensated for by a growing channel depth (more distinct learned patterns tracked per position), which is exactly the general CNN trade-off the CNN guide's Section 7 introduced: less spatial detail, richer per-position description. Only at the very last step - the classifier head - does C collapse hard, from 576 all the way down to 3, because that is the one place in the whole network whose entire job is to throw away everything except the final decision.

Initial convolution, worked with the formula. Input 224x224x3 passes through a standard 3x3 convolution with stride 2 and 'same'-style padding (P=1, following the CNN guide's Section 4.2 Case B logic). Using the formula: O = (224 - 3 + 2(1)) / 2 + 1 = (224 - 3 + 2)/2 + 1 = 223/2 + 1 = 111.5 + 1, floored to **112**. Output shape: 112x112x16 - H and W both dropped from the stride-2 formula; C became 16 simply because this stem layer was built with 16 filters.

Global average pooling, the H,W-to-1 step. Exactly the squeeze operation from Section 4.2, but applied here to collapse the entire spatial grid at once: the 7x7 spatial area of each of the 576 channels is averaged down to one single number per channel. Output shape: 1x1x576 - H and W collapse to exactly 1 (there is no more spatial grid left at all, only a list of 576 numbers), while C stays at 576 unchanged, since pooling never touches channel depth (CNN guide, Section 6).

Classifier head, the C-collapsing step. Two dense-equivalent layers (Section 7.3's derivation above) take the 576-number vector and, through Linear(576→1024) then Linear(1024→3), collapse C down to just 3 - the final fire/neutral/smoke logits. H and W never appear in this step at all (they are already fixed at 1x1, and stay there), since a fully-collapsed 1x1 spatial size behaves exactly like an ordinary flat vector feeding a dense layer (CNN guide, Section 8's flattening discussion).

This entire trace - 224x224x3 all the way down to a 3-number prediction vector - is the complete forward pass. Every single shape change along the way reduces to the same two independent controls: stride governs H and W, filter/output-channel count governs C, and every operation in Sections 2 through 5 of this document is built from exactly these two knobs.

Check your understanding: *using the rule from Section 8.3, if a hypothetical bottleneck block used stride 1 in its depthwise step but its projection step changed the channel count from 40 to 48, would H and W change, would C change, or would both change - and which specific sub-step inside the block (Section 3.2) is responsible for each answer?*

9. From One Layer's Output to the Next - How Channels Combine

9.1 The Question This Section Actually Answers

Section 8 traced how the spatial size and channel depth change, layer by layer, through the network. It stated, but never showed with real numbers, one specific claim: that a later layer's filter reaches across every channel produced by the layer before it, and combines them into something new - edges becoming curves, curves becoming textures, textures becoming object parts. This section proves that claim by hand, with a small, fully worked example small enough to compute every number, rather than leaving it as a plausible-sounding story.

9.2 Step 1 - Three Layer-1 Channels, Each Detecting Something Different

Take a tiny 5x5 patch of a real image containing something like a bright vertical line crossed by a shorter horizontal bar - a rough, simplified stand-in for a flame's stem crossed by a flicker of brighter color. Layer 1 does not have just one filter (Section 3 of this document) - suppose, for this worked example, it has three: Channel A, a vertical-edge detector; Channel B, a horizontal-edge detector; Channel C, a color-blob detector, tuned to the bright central region regardless of its shape. Each is a completely ordinary convolution (CNN guide, Section 2), just aimed at a different pattern.

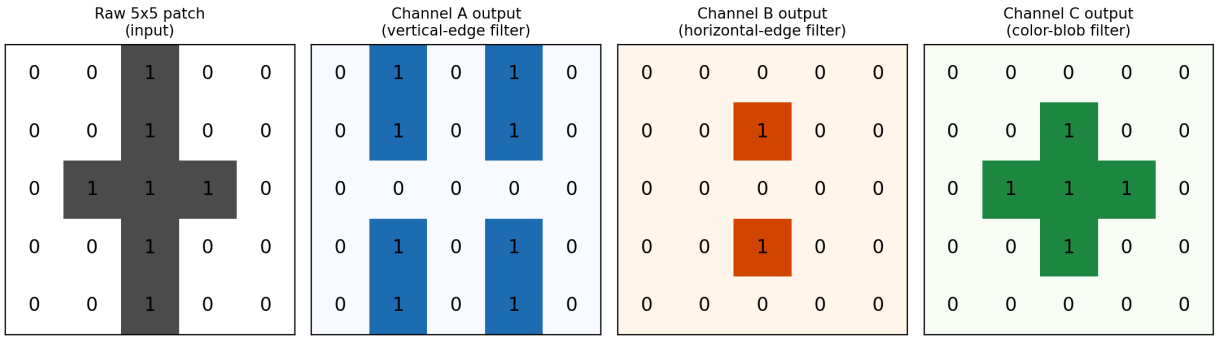


Figure 9.1 - One raw 5x5 patch produces three genuinely different channel outputs: Channel A lights up on the two vertical edges of the stripe, Channel B lights up on the horizontal bar's edges, and Channel C lights up in a cross-shaped region centered on the bright patch. Same input, three different questions being asked of it.

This is the literal, numeric version of what Section 3.1 stated in words: one filter can only look for one pattern. Here, three filters looked for three different patterns, on the exact same 5x5 patch, and produced three genuinely different 5x5 grids of numbers as a result. None of these three grids alone tells you 'this is a flame' - each is only answering its own narrow question (is there a vertical edge here? a horizontal edge? a bright blob?).

9.3 Step 2 - Layer 1's True Output Is a Stack, Not Three Separate Pictures

Layer 1's actual output, as covered in Section 3.2 of this document, is not three separate images - it is one single 5x5x3 block, a stack with Channel A, B, and C sitting on top of each other at every spatial position. At the exact center position of this 5x5 grid, for instance, the stack holds three numbers at once: Channel A's value there, Channel B's value there, Channel C's value there. This stacked block, in full, is what gets handed to layer 2 - never the three individual channel pictures in isolation, and never the original raw pixels again.

9.4 Step 3 - Layer 2's Filter Reaches Across All Three Channels at Once

A layer-2 filter that operates on this 5x5x3 stack is not a single 3x3 grid of 9 weights - exactly as Section 3.2 explained, its true shape is 3x3x3 (one 3x3 slice per input channel, 27 weights total), because it must span the full depth of whatever it is convolving over. Here is one such layer-2 filter, built by hand for this example so that it rewards a very specific higher-level combination: a vertical edge with a horizontal edge crossing it, centered on a bright blob - in other words, exactly a small cross- or plus-shaped hot region, which is a step closer to 'a bit of flickering flame' than any one of the three original patterns was on its own.

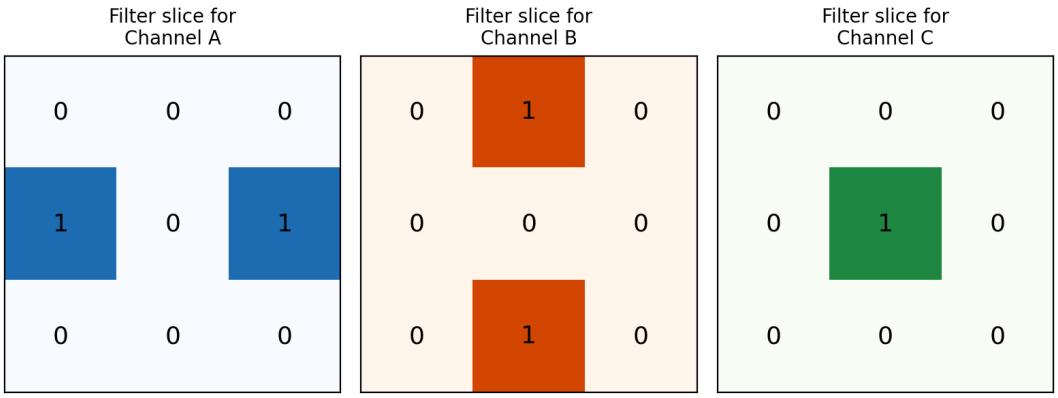


Figure 9.2 - One layer-2 filter's three 3x3 slices. The Channel-A slice rewards vertical edges appearing on the left and right of the window; the Channel-B slice rewards horizontal edges above and below; the Channel-C slice rewards a bright blob sitting exactly at the center. Together these three slices form ONE filter, 27 weights in total.

Let's convolve this filter over the exact center 3x3 window of Figure 9.1's three channels. Reading the actual windows straight from Figure 9.3: Channel A's window is [[1,0,1],[0,0,0],[1,0,1]] (the vertical edges pass through the left and right columns, nothing in the middle column at this position), Channel B's window is [[0,1,0],[0,0,0],[0,1,0]] (the horizontal edges from the bar sit in the top and bottom rows here), and Channel C's window is [[0,1,0],[1,1,1],[0,1,0]] (the color-blob channel's cross shape, centered). Multiplying each channel's window against its matching filter slice and summing, exactly as in the CNN guide's Section 2.2, but now doing this three times (once per channel) and adding all three channel-sums together into one single number:

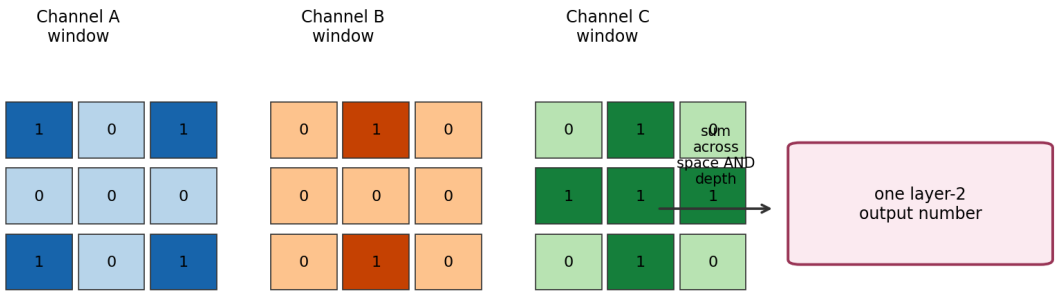


Figure 9.3 - Three separate windows, one from each channel, at the same spatial position - convolved against their matching filter slice, then summed across BOTH space (9 positions per slice) AND depth (3 slices) into a single layer-2 output number.

Channel A's contribution: filter slice A is [[0,0,0],[1,0,1],[0,0,0]] (rewards a vertical edge appearing in the middle row's left and right cells). Multiplying element-wise against the window above: every filter position that is 1 lands on a window value of 0 here (the window's middle row is all zeros), so Channel A contributes **0** at this exact position - this particular window simply does not have a vertical edge sitting where this filter slice is looking for one. Channel B's contribution: filter slice B is [[0,1,0],[0,0,0],[0,1,0]] (rewards a horizontal edge above and below). The window's top-middle and bottom-middle cells are both 1, exactly matching the filter's two active positions: (0x0 + 1x1 + 0x0) + (0x0+0x0+0x0) + (0x0 + 1x1 + 0x0) = 1 + 0 + 1 = **2**. Channel C's contribution: filter slice C is [[0,0,0],[0,1,0],[0,0,0]] (rewards a bright blob dead center). The window's exact center cell is 1: (0x0+1x0+0x0) + (1x0 + 1x1 + 1x0) + (0x0+1x0+0x0) = 0 + 1 + 0 = **1**. Summing across all three channels: 0 + 2 + 1 = **3**.

That single number, 3, is layer 2's output at this one spatial position - and it is a genuinely new quantity that did not exist anywhere in layer 1. It is not Channel A's value, not Channel B's, not Channel C's - it is a specific, weighted combination of all three at once. Notice Channel A contributed nothing at this exact position (no vertical edge happened to sit where that filter slice was looking), while Channel B and Channel C both contributed positively, because a horizontal edge and a bright blob genuinely did line up here. Slide this same 3x3x3 filter to a position where all three patterns line up together at once - a real vertical edge, a real horizontal edge, and a real bright blob, all simultaneously - and the output number would climb higher still, since every one of the three channel contributions would turn positive together.

9.5 Why This Is the Actual Mechanism Behind 'Edges Become Curves Become Textures'

Section 13.3 stated that deeper layers combine simple edges into curves, then textures, then object parts. The arithmetic above is exactly, literally what that sentence is describing - just at a scale of 3 channels and one filter instead of MobileNetV3-Small's real 16, 96, or 576 channels and hundreds of filters per stage (Section 7.2's architecture table). A real layer-2 filter never 'chooses' one channel to look at - the mathematics forces it to look at all of them simultaneously, at every single position, because a filter's shape (F x F x C, Section 3.2) always spans the full input depth. There is no way to build a

convolutional filter that reaches only some of the previous layer's channels; depth is never partial.

This is also precisely why a single feature-map snapshot (of the kind covered in the standalone 'Feature Map Image Explained' document) cannot, by itself, tell you 'this channel does edges, this one does texture' with certainty - what a channel encodes several layers in is genuinely an emergent combination of everything upstream of it, shaped jointly by every filter it passed through, not a clean, human-nameable single concept. The early layers in that document's reference image looked similar to each other precisely because they were all still close to raw pixels (little combination had happened yet); the late layers looked different from each other precisely because, by that depth, many rounds of exactly this cross-channel combination (as worked out numerically above) had already taken place.

Check your understanding: *if Channel A's filter slice in Figure 9.2 had instead been all zeros (a slice that ignores the vertical-edge channel entirely, contributing nothing no matter what Channel A's window contained), would the final layer-2 output number computed above (3) have changed - and what does your answer tell you about which channels a given layer-2 filter can effectively choose to ignore, even though it technically spans the full input depth?*

10. Bridging to FireWatch - Transfer Learning, Head Replacement, and Fine-Tuning

10.1 What Actually Changed From the Published Architecture

FireWatch's vision model is MobileNetV3-Small, ImageNet-pretrained, fine-tuned into a 3-class (neutral / smoke / fire) classifier - this exact model is FireWatch's production checkpoint 'v4' (models/fire_mnv3.onnx). Everything in Sections 1 through 8 of this document describes the architecture exactly as it came off the shelf. Only one structural change was made to it.

Recall from Section 8 that the stock architecture ends in a layer that projects 1024 channels down to 1000 ImageNet class scores. FireWatch's project report is explicit about this layer: it is 'Linear(1024 → 1000) for ImageNet's 1000 classes, which is meaningless for this task. That final layer is discarded and replaced with a fresh Linear(1024 → 3).' Every layer before it - every bottleneck block from Section 7.2's table, every squeeze-and-excite module, every h-swish activation - keeps its original ImageNet-pretrained weights untouched at first; only this new, randomly initialized 3-output head is added.

10.2 Why Transfer Learning Instead of Training From Scratch

The CNN guide's Section 13.3 explained that a network's filters start as random noise and gradually become meaningful pattern detectors purely through repeated exposure to a labeled training set. That process needs a lot of images to work reliably from scratch - FireWatch's project report states the training dataset is 18,678 images, and explicitly frames this as 'far too few to learn general visual features (edges, textures, colour gradients, shape) from scratch.' ImageNet, the dataset MobileNetV3-Small was originally pretrained on, contains roughly 1.2 million images - about 64 times more. Rather than trying to learn the general concept of 'an edge' or 'a color gradient' from 18,678 images, FireWatch starts from a network that has already learned those foundational visual features from 1.2 million images, and only needs to learn the much narrower, final step: how to map those already-good features onto the specific categories neutral, smoke, and fire.

10.3 The Two-Stage Training Schedule

FireWatch's report describes training in exactly two stages over 12 total epochs (one epoch = one full pass through the training set, CNN guide Section 13.2, Step 6).

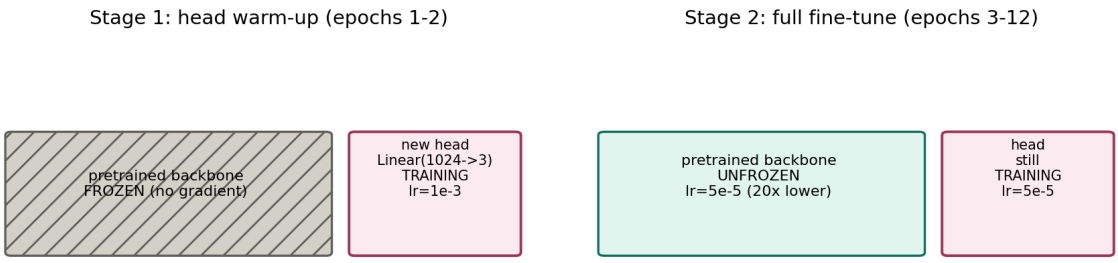


Figure 9.1 - FireWatch's two-stage fine-tuning schedule. Stage 1 trains only the new head while the pretrained backbone stays frozen; Stage 2 unfreezes everything, but at a much lower learning rate.

Stage 1 (epochs 1-2, learning rate 1e-3). Only the new Linear(1024 → 3) head is trainable; every other layer's weights are frozen (requires_grad=False in the code, meaning backpropagation, CNN guide Section 11, still computes gradients flowing through those layers on the way back to the head, but never actually updates their weights via gradient descent, CNN guide Section 12). The project's reasoning, direct from the report: 'the random head's large, noisy early gradients would otherwise backpropagate into and scramble the pretrained filters before the head has learned anything sensible.' This connects directly to backpropagation's mechanics from the CNN guide - a freshly initialized layer produces large, essentially random gradients at first (since its weights have not yet learned anything meaningful), and letting those large, noisy gradient-descent updates reach deep into the carefully pretrained backbone weights would partially undo the useful structure ImageNet pretraining had already built.

Stage 2 (epochs 3-12, learning rate 5e-5 - twenty times lower than Stage 1). Every parameter in the entire network is unfrozen and becomes trainable, including every bottleneck block's weights throughout the whole architecture in Section 7.2's table. The report's stated reasoning: 'once the head is sensible, the pretrained ImageNet filters are nudged gently toward fire/smoke-specific textures. A stage-1-sized learning rate here would overwrite useful pretrained weights instead of adapting them - catastrophic forgetting.' This connects directly to the CNN guide's Section 12.3: too large a learning rate risks overshooting a good minimum. Here, the specific risk of a large learning rate is not just numerical overshoot but actively destroying good pretrained representations that took 1.2 million ImageNet images to learn in the first place - so a much smaller step size is used to nudge everything gently rather than overwrite it wholesale.

10.4 The Input Pipeline - Why It Must Match Pretraining Exactly

FireWatch's inputs are 224x224 RGB - the exact input size this whole document has traced through the network (Section 8). Critically, the project report specifies inputs are 'normalized with ImageNet's channel statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225])' and explains why: 'the pretrained filters were themselves trained on inputs normalized this exact way; anything else would push inputs out of the distribution the pretrained features expect.' This is a direct, practical consequence of how every filter in Section 7's table was originally trained - a filter's learned weights are only meaningful relative to the specific numeric range of inputs it saw during its original training. Feeding it pixel values normalized a different way is a bit like handing a thermometer calibrated in Celsius a reading it's told is Fahrenheit - the filter's weights would be reacting to numbers on a scale they were never tuned for.

10.5 Export and the Decision Layer on Top of the Raw Softmax Output

After training, the model is exported from PyTorch to ONNX for the edge loop's inference path, with every promoted checkpoint verified for numerical equivalence between the two formats (v4's maximum logit difference was 5.99 x 10^-5, against a 1x10^-4 pass bar) - confirming the exported model computes the exact same forward pass described in Section 8, just in a different runtime. One further layer sits on top of the raw 3-class softmax output (CNN guide Section 5.6) before FireWatch calls a final verdict: fire is only declared if P(fire) is at least 0.30, and smoke requires winning the 3-way vote **and** clearing P(smoke) of at least 0.45 - a rule added after live testing showed smoke could spuriously win the 3-way argmax at lower confidence levels. Neither of these thresholds is part of the MobileNetV3-Small architecture itself - they are a decision layer FireWatch adds on top of the model's raw probability outputs, tuned specifically against this project's own precision and recall requirements.

11. Why MobileNetV3-Small Was the Right Choice for FireWatch

11.1 The Actual Constraint - Real-Time, on a CPU, Alongside Other Work

FireWatch's deployment target is a laptop CPU running ONNX Runtime at a minimum of 5 frames per second, while a live camera loop and the rest of the sensor-fusion pipeline are also running simultaneously on the same machine - there is no dedicated GPU to offload the model onto. This is precisely the deployment scenario every architectural choice in this document was built around: depthwise separable convolution (Section 2) to cut computation roughly 8-9x per layer, the inverted bottleneck (Section 3) to keep parameter-heavy operations narrow, h-swish (Section 5) chosen specifically to avoid expensive exponentials on constrained hardware, and the whole architecture shaped by NAS (Section 6) explicitly searching for strong accuracy at low measured latency on mobile-class hardware.

11.2 The Explicit Trade-off, in the Project's Own Words

FireWatch's report states this trade-off directly: MobileNetV3-Small 'is ~2.5M parameters and was designed via hardware-aware neural architecture search specifically for fast mobile/CPU inference - a ResNet-50-class model (~25M parameters) might add a point or two of raw accuracy but would risk the FPS budget the whole safety case depends on. This is a deliberate params-vs-latency trade: give up a small amount of ceiling accuracy to guarantee the 30 FPS the temporal voter and buzzer latency targets assume.' This sentence directly connects the parameter-count math worked through in Sections 2, 3, and 7.3 of this document (2.5 million versus roughly 25 million, about a 10x difference) to a concrete safety consequence: FireWatch's temporal voter (which requires 5 of 8 consecutive frames to agree before trusting a fire signal) and its buzzer response latency both assume the model can keep up with a live camera feed. A larger, slower model - even one with slightly higher raw accuracy on a static benchmark - would risk breaking that real-time assumption entirely, which matters more for a safety system than a marginal accuracy gain.

11.3 Why Transfer Learning Specifically, Not Just a Small Custom Network

It is worth being precise about two separate decisions that are easy to conflate: choosing MobileNetV3-Small's architecture (a speed decision, Section 10.1-10.2), and choosing to fine-tune ImageNet-pretrained weights rather than training that same architecture from random initialization (a data-efficiency decision, Section 9.2). The project's report lists this as its own explicit design change: 'Train a CNN from scratch' was the original plan, revised to 'Fine-tune pretrained MobileNetV3-Small,' with the stated reason: 'No time to collect and label a dedicated fire dataset; transfer learning reaches useful accuracy in one session.' Both decisions point the same direction - toward a widely available, pre-existing, well-tested architecture and weight set, rather than designing or training something bespoke - because bespoke solutions on both axes (architecture search and large-scale pretraining) require resources (compute time, dataset size) well beyond what a single project session has available.

11.4 The Measured Result

The production checkpoint (v4) reaches 0.9575 fire recall and 0.8511 smoke recall on a leak-free validation split, and a live 12-trial evaluation passed 11 of 12 trials. These are the real, measured consequences of every decision traced through this document: the architecture (Sections 1-8) made real-time CPU inference possible at all; transfer learning (Section 9) made reaching this accuracy achievable within one project's data and time budget; and the two-stage fine-tuning schedule (Section 9.3) is what actually adapted the pretrained network's general visual knowledge onto this specific fire/smoke/neutral task without destroying it in the process.

12. Quick Reference - Formulas, Shapes, and Vocabulary

12.1 Core Formulas

Concept	Formula
Depthwise convolution parameters	F x F x C
Pointwise (1x1) convolution parameters	C x K
Total depthwise separable parameters	(F x F x C) + (C x K)
Standard convolution parameters (for comparison)	F x F x C x K
Global average pooling (squeeze)	average of all H x W values, per channel
h-sigmoid	clip((x + 3) / 6, 0, 1)
h-swish	x * h-sigmoid(x)
Output spatial size (from the CNN guide)	O = (N - F + 2P) / S + 1

12.2 Vocabulary Glossary

Depthwise convolution - Each input channel is convolved with its own separate spatial filter, never mixing with other channels.

Pointwise convolution - A 1x1 convolution that mixes channels together without looking at any spatial neighbors.

Depthwise separable convolution - A depthwise step followed by a pointwise step, approximating a standard convolution far more cheaply.

Inverted residual bottleneck - A block that expands channels, filters spatially, then projects back down to a narrow width, with an optional skip connection.

Expansion ratio - How many times wider the intermediate (expanded) representation is compared to the block's narrow input/output width.

Linear bottleneck - The final projection step of a bottleneck block uses no activation function, to avoid destroying information in a narrow channel space.

Residual / skip connection - Adding a block's original input directly to its output, giving gradients a direct backward path; only used when shapes match.

Squeeze-and-excite (SE) - A side-mechanism that computes per-channel importance weights and rescales a feature map's channels accordingly.

Global average pooling - Averaging an entire spatial feature map down to a single number per channel.

Swish - The smooth activation function x times sigmoid(x).

h-swish / h-sigmoid - Cheap, piecewise-linear approximations of swish and sigmoid, avoiding expensive exponentials.

Neural Architecture Search (NAS) - An automated search process that proposes and evaluates many candidate architectures against real hardware measurements.

Transfer learning - Reusing a network already trained on a large dataset as the starting point for a new, related task.

Fine-tuning - Continuing to train a pretrained network's weights (all or some of them) on new, task-specific data.

Frozen layer - A layer whose weights are excluded from gradient descent updates during a given training stage, even though gradients still flow through it during backpropagation.

Catastrophic forgetting - A pretrained network's useful weights being overwritten and destroyed by too-large gradient updates during fine-tuning.

ONNX - An open, framework-independent format for exporting a trained model so it can run in a different inference runtime than the one it was trained in.

End of guide. This document assumed the companion CNN guide throughout - if any concept here felt unfamiliar rather than just new, it is worth checking whether it was actually covered in the prerequisites recap (Section 0) or the CNN guide itself before continuing further into MobileNetV3 or FireWatch-specific material.